

Kinematics and Odometry

运动学与里程计

中英对照 · Java 文档 · 7 篇文章

Contents / 目录

01 Introduction to Kinematics and The ChassisSpeeds Class / 运动学与 ChassisSpeeds 类简介	2
02 Differential Drive Kinematics / 差速驱动运动学	4
03 Differential Drive Odometry / 差速驱动里程计	5
04 Swerve Drive Kinematics / Swerve Drive 运动学	7
05 Swerve Drive Odometry / Swerve Drive 里程计	12
06 Mecanum Drive Kinematics / 麦克纳姆驱动运动学	14
07 Mecanum Drive Odometry / 麦克纳姆驱动里程计	16

01 Introduction to Kinematics and The ChassisSpeeds Class / 运动学与 ChassisSpeeds 类简介

NOTE: Kinematics and odometry uses a common coordinate system. You may wish to reference the [coordinate system](#) section for details.

说明： 运动学和里程计使用共同的坐标系。有关详细信息，可参阅坐标系章节。

What is kinematics? / 什么是运动学？

The kinematics suite contains classes for differential drive, swerve drive, and mecanum drive kinematics and odometry. The kinematics classes help convert between a universal `ChassisSpeeds` ([Java](#)) object, containing linear and angular velocities for a robot, and usable speeds for each individual type of drivetrain i.e. left and right wheel speeds for a differential drive, four wheel speeds for a mecanum drive, or individual module states (speed and angle) for a swerve drive.

运动学套件包含差速驱动、swerve drive 和麦克纳姆驱动的运动学与里程计类。运动学类可在通用的 `ChassisSpeeds` ([Java](#)) 对象与各类底盘可用的速度之间转换。该对象包含机器人的线速度和角速度；转换后的结果例如差速驱动的左右轮速度、麦克纳姆驱动的四轮速度，或 swerve drive 各模块的状态（速度和角度）。

What is odometry? / 什么是里程计？

Odometry involves using sensors on the robot to create an estimate of the position of the robot on the field. In FRC, these sensors are typically several encoders (the exact number depends on the drive type) and a gyroscope to measure robot angle. The odometry classes utilize the kinematics classes along with periodic user inputs about speeds (and angles in the case of swerve) to create an estimate of the robot's location on the field.

里程计 (odometry) 利用机器人上的传感器估计机器人在场地上的位置。在 FRC 中，这些传感器通常是若干编码器（具体数量取决于底盘类型）和用于测量机器人角度的陀螺仪。里程计类结合运动学类，以及用户周期性提供的速度信息（swerve drive 还需角度信息），估计机器人在场地上的位置。

The ChassisSpeeds Class / ChassisSpeeds 类

The `ChassisSpeeds` object is essential to the new WPILib kinematics and odometry suite. The `ChassisSpeeds` object represents the speeds of a robot chassis. This struct has three components:

`ChassisSpeeds` 对象是 WPILib 运动学与里程计套件的核心。它表示机器人底盘的速度，包含三个分量：

- `vx`: The velocity of the robot in the x (forward) direction.
- `vx`: 机器人沿 x 方向（向前）的速度。
- `vy`: The velocity of the robot in the y (sideways) direction. (Positive values mean the robot is moving to the left).
- `vy`: 机器人沿 y 方向（侧向）的速度。（正值表示机器人向左移动。）
- `omega`: The angular velocity of the robot in radians per second.
- `omega`: 机器人的角速度，单位为弧度每秒。

NOTE: A non-holonomic drivetrain (i.e. a drivetrain that cannot move sideways, ex: a differential drive) will have a v_y component of zero because of its inability to move sideways.

说明： 非全向底盘（即无法侧向移动的底盘，例如差速驱动）的 v_y 分量为零，因为它不能侧向移动。

Constructing a ChassisSpeeds object / 构造 ChassisSpeeds 对象

The constructor for the `ChassisSpeeds` object is very straightforward, accepting three arguments for v_x , v_y , and ω . In Java, v_x and v_y must be in meters per second.

`ChassisSpeeds` 对象的构造函数很直接，接收 v_x 、 v_y 和 ω 三个参数。在 Java 中， v_x 和 v_y 必须以米每秒为单位。

```
// The robot is moving at 3 meters per second forward, 2 meters
// per second to the right, and rotating at half a rotation per
// second counterclockwise.
var speeds = new ChassisSpeeds(3.0, -2.0, Math.PI);
```

Creating a ChassisSpeeds Object from Field-Relative Speeds / 根据场地相对速度创建 ChassisSpeeds 对象

A `ChassisSpeeds` object can also be created from a set of field-relative speeds when the robot angle is given. This converts a set of desired velocities relative to the field (for example, toward the opposite alliance station and toward the right field boundary) to a `ChassisSpeeds` object which represents speeds that are relative to the robot frame. This is useful for implementing field-oriented controls for a swerve or mecanum drive robot.

给定机器人角度时，也可以由一组相对于场地的速度创建 `ChassisSpeeds` 对象。这样会把相对于场地的目标速度（例如朝向对方联盟站、朝向场地右侧边界）转换为表示机器人坐标系下速度的 `ChassisSpeeds` 对象。这有助于实现 swerve drive 或麦克纳姆驱动机器人的场地定向控制。

The static `ChassisSpeeds.fromFieldRelativeSpeeds` method can be used to generate the `ChassisSpeeds` object from field-relative speeds. This method accepts the v_x (relative to the field), v_y (relative to the field), ω , and the robot angle.

静态方法 `ChassisSpeeds.fromFieldRelativeSpeeds` 可根据场地相对速度生成 `ChassisSpeeds` 对象。该方法接收 v_x （相对于场地）、 v_y （相对于场地）、 ω 和机器人角度。

```
// The desired field relative speed here is 2 meters per second
// toward the opponent's alliance station wall, and 2 meters per
// second toward the left field boundary. The desired rotation
// is a quarter of a rotation per second counterclockwise. The current
// robot angle is 45 degrees.
ChassisSpeeds speeds = ChassisSpeeds.fromFieldRelativeSpeeds(
    2.0, 2.0, Math.PI / 2.0, Rotation2d.fromDegrees(45.0));
```

NOTE: The angular velocity is not explicitly stated to be "relative to the field" because the angular velocity is the same as measured from a field perspective or a robot perspective.

说明： 这里没有明确将角速度称为“相对于场地”，因为从场地或机器人视角测得的角速度相同。

02 Differential Drive Kinematics / 差速驱动运动学

The `DifferentialDriveKinematics` class is a useful tool that converts between a `ChassisSpeeds` object and a `DifferentialDriveWheelSpeeds` object, which contains velocities for the left and right sides of a differential drive robot.

`DifferentialDriveKinematics` 类用于在 `ChassisSpeeds` 对象与 `DifferentialDriveWheelSpeeds` 对象之间转换；后者包含差速驱动机器人左右两侧的速度。

Constructing the Kinematics Object / 构造运动学对象

The `DifferentialDriveKinematics` object accepts one constructor argument, which is the track width of the robot. This represents the distance between the two sets of wheels on a differential drive.

`DifferentialDriveKinematics` 对象的构造函数接收一个参数：机器人的轮距。轮距表示差速驱动两侧车轮组之间的距离。

NOTE: In Java, the track width must be in meters.

说明：在 Java 中，轮距必须以米为单位。

Converting Chassis Speeds to Wheel Speeds / 将底盘速度转换为车轮速度

The `toWheelSpeeds(ChassisSpeeds speeds)` method should be used to convert a `ChassisSpeeds` object to a `DifferentialDriveWheelSpeeds` object. This is useful in situations where you have to convert a linear velocity (v_x) and an angular velocity (ω) to left and right wheel velocities.

应使用 `toWheelSpeeds(ChassisSpeeds speeds)` 方法将 `ChassisSpeeds` 对象转换为 `DifferentialDriveWheelSpeeds` 对象。需要把线速度 (v_x) 和角速度 (ω) 转换为左右轮速度时，该方法很有用。

```
// Creating my kinematics object: track width of 27 inches
DifferentialDriveKinematics kinematics =
    new DifferentialDriveKinematics(Units.inchesToMeters(27.0));
// Example chassis speeds: 2 meters per second linear velocity,
// 1 radian per second angular velocity.
var chassisSpeeds = new ChassisSpeeds(2.0, 0, 1.0);
// Convert to wheel speeds
DifferentialDriveWheelSpeeds wheelSpeeds = kinematics.toWheelSpeeds(chassisSpeeds);
// Left velocity
double leftVelocity = wheelSpeeds.leftMetersPerSecond;
// Right velocity
double rightVelocity = wheelSpeeds.rightMetersPerSecond;
```

Converting Wheel Speeds to Chassis Speeds / 将车轮速度转换为底盘速度

One can also use the kinematics object to convert individual wheel speeds (left and right) to a singular `ChassisSpeeds` object. The `toChassisSpeeds(DifferentialDriveWheelSpeeds speeds)` method should be used to achieve this.

也可以使用运动学对象，将左右轮各自的速度转换为单个 `ChassisSpeeds` 对象。应使用 `toChassisSpeeds(DifferentialDriveWheelSpeeds speeds)` 方法完成该转换。

```
// Creating my kinematics object: track width of 27 inches
DifferentialDriveKinematics kinematics =
    new DifferentialDriveKinematics(Units.inchesToMeters(27.0));
// Example differential drive wheel speeds: 2 meters per second
// for the left side, 3 meters per second for the right side.
var wheelSpeeds = new DifferentialDriveWheelSpeeds(2.0, 3.0);
// Convert to chassis speeds.
ChassisSpeeds chassisSpeeds = kinematics.toChassisSpeeds(wheelSpeeds);
// Linear velocity
double linearVelocity = chassisSpeeds.vxMetersPerSecond;
// Angular velocity
double angularVelocity = chassisSpeeds.omegaRadiansPerSecond;
```

03 Differential Drive Odometry / 差速驱动里程计

A user can use the differential drive kinematics classes in order to perform [odometry](#). WPILib contains a `DifferentialDriveOdometry` class that can be used to track the position of a differential drive robot on the field.

用户可以使用差速驱动运动学类进行[里程计计算](#)。WPILib 提供 `DifferentialDriveOdometry` 类，用于跟踪差速驱动机器人在场地上的位置。

NOTE: Because this method only uses encoders and a gyro, the estimate of the robot's position on the field will drift over time, especially as your robot comes into contact with other robots during gameplay. However, odometry is usually very accurate during the autonomous period.

说明： 由于这种方法只使用编码器和陀螺仪，对机器人场地位置的估计会随时间漂移；在比赛中机器人与其他机器人发生碰撞时尤为明显。不过，在自动阶段，里程计通常非常准确。

Creating the Odometry Object / 创建里程计对象

The `DifferentialDriveOdometry` class constructor requires three mandatory arguments and one optional argument. The mandatory arguments are:

`DifferentialDriveOdometry` 类的构造函数需要三个必选参数和一个可选参数。必选参数为：

- The angle reported by your gyroscope (as a `Rotation2d`)
- 陀螺仪报告的角度（表示为 `Rotation2d`）。
- The initial left and right encoder readings. In Java, these are a number that represents the distance traveled by each side in meters.
- 左右编码器的初始读数。在 Java 中，这些读数是表示每侧行驶距离的数值，单位为米。

The optional argument is the starting pose of your robot on the field (as a `Pose2d`). By default, the robot will start at $x = 0$, $y = 0$, $\theta = 0$.

可选参数是机器人在场地上的初始位姿（pose，以 `Pose2d` 表示）。默认情况下，机器人从 $x = 0$, $y = 0$, $\theta = 0$ 开始。

NOTE: 0 degrees / radians represents the robot angle when the robot is facing directly toward your opponent's alliance station. As your robot turns to the left, your gyroscope angle should increase. The `Gyro` interface supplies `getRotation2d` that you can use for this purpose. See [coordinate system](#) for more information about the coordinate system.

说明： 0 度 / 弧度表示机器人正对方联盟站时的朝向。机器人向左转动时，陀螺仪角度应增大。`Gyro` 接口提供可用于此目的的 `getRotation2d`。有关坐标系的更多信息，请参阅坐标系章节。

```
// Creating my odometry object. Here,
// our starting pose is 5 meters along the long end of the field and in the
// center of the field along the short end, facing forward.
DifferentialDriveOdometry m_odometry = new DifferentialDriveOdometry(
    m_gyro.getRotation2d(),
    m_leftEncoder.getDistance(), m_rightEncoder.getDistance(),
    new Pose2d(5.0, 13.5, new Rotation2d()));
```

Updating the Robot Pose / 更新机器人位姿

The `update` method can be used to update the robot's position on the field. This method must be called periodically, preferably in the `periodic()` method of a [Subsystem](#). The `update` method returns the new updated pose of the robot. This method takes in the gyro angle of the robot, along with the left encoder distance and right encoder distance.

`update` 方法用于更新机器人在场地上的位置。必须周期性调用此方法，最好在某个 [Subsystem](#) 的 `periodic()` 方法中调用。`update` 方法返回更新后的机器人位姿。它接收机器人陀螺仪角度以及左右编码器的距离。

NOTE: If the robot is moving forward in a straight line, **both** distances (left and right) must be increasing positively -- the rate of change must be positive.

说明： 如果机器人沿直线向前移动，左右**两侧**距离都必须正向增加，即变化率必须为正。

```
@Override
public void periodic() {
    // Get the rotation of the robot from the gyro.
    var gyroAngle = m_gyro.getRotation2d();
    // Update the pose
    m_pose = m_odometry.update(gyroAngle,
        m_leftEncoder.getDistance(),
        m_rightEncoder.getDistance());
}
```

Resetting the Robot Pose / 重置机器人位姿

The robot pose can be reset via the `resetPosition` method. This method accepts four arguments: the current gyro angle, the left and right wheel positions, and the new field-relative pose.

可以通过 `resetPosition` 方法重置机器人位姿。该方法接收四个参数：当前陀螺仪角度、左右轮位置，以及新的场地相对位姿。

IMPORTANT: If at any time, you decide to reset your gyroscope or encoders, the `resetPosition` method **MUST** be called with the new gyro angle and wheel distances.

重要： 如果你在任何时候重置陀螺仪或编码器，**必须**使用新的陀螺仪角度和车轮距离调用 `resetPosition` 方法。

NOTE: A full example of a differential drive robot with odometry is available here: [Java](#)

说明： 这里提供一个带里程计的差速驱动机器人完整示例：[Java](#)。

In addition, the `getPoseMeters` method can be used to retrieve the current robot pose without an update.

此外，还可以使用 `getPoseMeters` 方法在不更新里程计的情况下获取机器人当前位姿。

04 Swerve Drive Kinematics / Swerve Drive 运动学

The `SwerveDriveKinematics` class is a useful tool that converts between a `ChassisSpeeds` object and several `SwerveModuleState` objects, which contains velocities and angles for each swerve module of a swerve drive robot.

`SwerveDriveKinematics` 类用于在 `ChassisSpeeds` 对象与多个 `SwerveModuleState` 对象之间转换；每个模块状态包含 swerve drive 机器人对应模块的速度和角度。

NOTE: Swerve drive kinematics uses a common coordinate system. You may wish to reference the [coordinate system](#) section for details.

说明： swerve drive 运动学使用共同的坐标系。有关详细信息，可参阅坐标系章节。

The swerve module state class / Swerve 模块状态类

The `SwerveModuleState` class contains information about the velocity and angle of a singular module of a swerve drive. The constructor for a `SwerveModuleState` takes in two arguments, the velocity of the wheel on the module, and the angle of the module.

`SwerveModuleState` 类保存 swerve drive 单个模块的速度和角度信息。其构造函数接收两个参数：模块车轮的速度和模块角度。

NOTE: In Java, the velocity of the wheel must be in meters per second.

说明： 在 Java 中，车轮速度必须以米每秒为单位。

NOTE: An angle of 0 corresponds to the modules facing forward.

说明： 角度 0 对应模块朝向机器人前方。

Constructing the kinematics object / 构造运动学对象

The `SwerveDriveKinematics` class accepts a variable number of constructor arguments, with each argument being the location of a swerve module relative to the robot center (as a `Translation2d`). The number of constructor arguments corresponds to the number of swerve modules.

`SwerveDriveKinematics` 类的构造函数可接收可变数量的参数，每个参数都是一个 swerve 模块相对于机器人中心的位置（以 `Translation2d` 表示）。参数数量与 swerve 模块数量相同。

NOTE: A swerve drive must have 2 or more modules.

说明： swerve drive 必须至少有 2 个模块。

The locations for the modules must be relative to the center of the robot. Positive x values represent moving toward the front of the robot whereas positive y values represent moving toward the left of the robot.

模块位置必须相对于机器人中心。x 为正表示朝机器人前方，y 为正表示朝机器人左方。


```
// Locations for the swerve drive modules relative to the robot center.
Translation2d m_frontLeftLocation = new Translation2d(0.381, 0.381);
Translation2d m_frontRightLocation = new Translation2d(0.381, -0.381);
Translation2d m_backLeftLocation = new Translation2d(-0.381, 0.381);
Translation2d m_backRightLocation = new Translation2d(-0.381, -0.381);
// Creating my kinematics object using the module locations
SwerveDriveKinematics m_kinematics = new SwerveDriveKinematics(
    m_frontLeftLocation, m_frontRightLocation, m_backLeftLocation, m_backRightLocation
);
```

Converting chassis speeds to module states / 将底盘速度转换为模块状态

The `toSwerveModuleStates(ChassisSpeeds speeds)` method should be used to convert a `ChassisSpeeds` object to an array of `SwerveModuleState` objects. This is useful in situations where you have to convert a forward velocity, sideways velocity, and an angular velocity into individual module states.

应使用 `toSwerveModuleStates(ChassisSpeeds speeds)` 方法将 `ChassisSpeeds` 对象转换为 `SwerveModuleState` 对象数组。需要把前向速度、侧向速度和角速度转换为各模块状态时，该方法很有用。

The elements in the array that is returned by this method are the same order in which the kinematics object was constructed. For example, if the kinematics object was constructed with the front left module location, front right module location, back left module location, and the back right module location in that order, the elements in the array would be the front left module state, front right module state, back left module state, and back right module state in that order.

该方法返回的数组元素顺序与构造运动学对象时传入的模块顺序相同。例如，如果构造时依次传入左前、右前、左后、右后模块的位置，则数组中的元素也依次是左前、右前、左后、右后模块的状态。

```
// Example chassis speeds: 1 meter per second forward, 3 meters
// per second to the left, and rotation at 1.5 radians per second
// counterclockwise.
ChassisSpeeds speeds = new ChassisSpeeds(1.0, 3.0, 1.5);
// Convert to module states
SwerveModuleState[] moduleStates = kinematics.toSwerveModuleStates(speeds);
// Front left module state
SwerveModuleState frontLeft = moduleStates[0];
// Front right module state
SwerveModuleState frontRight = moduleStates[1];
// Back left module state
SwerveModuleState backLeft = moduleStates[2];
// Back right module state
SwerveModuleState backRight = moduleStates[3];
```

Module angle optimization / 模块角度优化

The `SwerveModuleState` class contains a static `optimize()` method that is used to "optimize" the speed and angle setpoint of a given `SwerveModuleState` to minimize the change in heading. For example, if the angular setpoint of a certain module from inverse kinematics is 90 degrees, but your current angle is -89 degrees, this method will automatically negate the speed of the module setpoint and make the angular setpoint -90 degrees to reduce the distance the module has to travel.

`SwerveModuleState` 类提供静态方法 `optimize()`，用于“优化”给定 `SwerveModuleState` 的速度与角度 setpoint，以尽量减小模块朝向的变化。例如，逆运动学给出的某模块目标角度为 90 度，而当前角度为 -89 度时，该方法会自动将模块速度 setpoint 取反，并把目标角度改为 -90 度，从而缩短模块需要转动的角度。

This method takes two parameters: the desired state (usually from the `toSwerveModuleStates` method) and the current angle. It will return the new optimized state which you can use as the setpoint in your feedback control loop.

该方法接收两个参数：期望状态（通常来自 `toSwerveModuleStates` 方法）和当前角度。它返回优化后的新状态，可将其用作反馈控制环的 setpoint。

```
var frontLeftOptimized = SwerveModuleState.optimize(frontLeft,
    new Rotation2d(m_turningEncoder.getDistance()));
```

Cosine compensation / 余弦补偿

Cosine compensation is a technique that reduces the speed of a module when it is not pointing in the desired direction. This is done by multiplying the desired speed of the module by the cosine of the angle error.

余弦补偿是一种在模块未指向期望方向时降低其速度的技术。具体做法是将模块的目标速度乘以角度误差的余弦值。

- If the wheel is pointing straight in the desired direction, then the speed remains unchanged as $\cos(0^\circ) = 1$.
- 如果车轮正好指向期望方向，速度保持不变，因为 $\cos(0^\circ) = 1$ 。
- If the wheel is perpendicular to the desired direction of motion, then the speed is reduced to 0 as $\cos(90^\circ) = 0$.
- 如果车轮与期望运动方向垂直，速度降至 0，因为 $\cos(90^\circ) = 0$ 。
- Everything in between follows the cosine curve.
- 其间的情况遵循余弦曲线。

Cosine compensation has been shown to reduce the amount of "skew" a swerve drive experiences when changing direction.

已有结果表明，余弦补偿可以减少 swerve drive 在改变方向时出现的“偏斜”。

```
var currentAngle = new Rotation2d.fromRadians(m_turningEncoder.getDistance());
var frontLeftOptimized = SwerveModuleState.optimize(frontLeft, currentAngle);
frontLeftOptimized.speedMetersPerSecond *= frontLeftOptimized.angle.minus(currentAngle).getCos();
```

译注：原文 Java 示例中的 `new Rotation2d.fromRadians(...)` 不是合法的 Java 调用形式；此处按纯翻译要求保留原代码。

Field-oriented drive / 场地定向驾驶

[Recall](#) that a `ChassisSpeeds` object can be created from a set of desired field-oriented speeds. This feature can be used to get module states from a set of desired field-oriented speeds.

[回顾](#)：可以由一组期望的场地定向速度创建 `ChassisSpeeds` 对象。利用这一功能，可以根据一组期望的场地定向速度得到模块状态。

```
// The desired field relative speed here is 2 meters per second
// toward the opponent's alliance station wall, and 2 meters per
// second toward the left field boundary. The desired rotation
// is a quarter of a rotation per second counterclockwise. The current
// robot angle is 45 degrees.
ChassisSpeeds speeds = ChassisSpeeds.fromFieldRelativeSpeeds(
    2.0, 2.0, Math.PI / 2.0, Rotation2d.fromDegrees(45.0));
// Now use this in our kinematics
SwerveModuleState[] moduleStates = kinematics.toSwerveModuleStates(speeds);
```

Using custom centers of rotation / 使用自定义旋转中心

Sometimes, rotating around one specific corner might be desirable for certain evasive maneuvers. This type of behavior is also supported by the WPILib classes. The same `toSwerveModuleStates()` method accepts a second parameter for the center of rotation (as a `Translation2d`). Just like the wheel locations, the `Translation2d` representing the center of rotation should be relative to the robot center.

在某些规避动作中，可能希望机器人绕某个特定角部旋转。WPIlib 类也支持这种行为。同一个 `toSwerveModuleStates()` 方法可接收第二个参数，表示旋转中心（类型为 `Translation2d`）。与车轮位置一样，表示旋转中心的 `Translation2d` 应相对于机器人中心。

NOTE: Because all robots are a rigid frame, the provided `vx` and `vy` velocities from the `ChassisSpeeds` object will still apply for the entirety of the robot. However, the `omega` from the `ChassisSpeeds` object will be measured from the center of rotation.

说明： 由于整个机器人是刚性框架，`ChassisSpeeds` 对象给出的 `vx` 和 `vy` 速度仍适用于整个机器人。不过，`ChassisSpeeds` 对象中的 `omega` 将以旋转中心为基准测量。

For example, one can set the center of rotation on a certain module and if the provided `ChassisSpeeds` object has a `vx` and `vy` of zero and a non-zero `omega`, the robot will appear to rotate around that particular swerve module.

例如，可以将旋转中心设在某个模块上；如果给定的 `ChassisSpeeds` 对象中 `vx` 和 `vy` 均为零，而 `omega` 非零，机器人看起来就会绕该 swerve 模块旋转。

Converting module states to chassis speeds / 将模块状态转换为底盘速度

One can also use the kinematics object to convert an array of `SwerveModuleState` objects to a singular `ChassisSpeeds` object. The `toChassisSpeeds(SwerveModuleState... states)` method can be used to achieve this.

也可以使用运动学对象，将 `SwerveModuleState` 对象数组转换为单个 `ChassisSpeeds` 对象。可使用 `toChassisSpeeds(SwerveModuleState... states)` 方法完成该转换。

```
// Example module states
var frontLeftState = new SwerveModuleState(23.43, Rotation2d.fromDegrees(-140.19));
var frontRightState = new SwerveModuleState(23.43, Rotation2d.fromDegrees(-39.81));
var backLeftState = new SwerveModuleState(54.08, Rotation2d.fromDegrees(-109.44));
var backRightState = new SwerveModuleState(54.08, Rotation2d.fromDegrees(-70.56));
// Convert to chassis speeds
ChassisSpeeds chassisSpeeds = kinematics.toChassisSpeeds(
    frontLeftState, frontRightState, backLeftState, backRightState);
// Getting individual speeds
double forward = chassisSpeeds.vxMetersPerSecond;
double sideways = chassisSpeeds.vyMetersPerSecond;
double angular = chassisSpeeds.omegaRadiansPerSecond;
```

Module state visualization with AdvantageScope / 使用 AdvantageScope 可视化模块状态

By recording a set of swerve module states using [NetworkTables](#) or [WPIlib data logs](#), [AdvantageScope](#) can be used to visualize the state of a swerve drive. The code below shows how a set of `SwerveModuleState` objects can be published to `NetworkTables`.

将一组 swerve 模块状态记录到 [NetworkTables](#) 或 [WPIlib 数据日志](#) 后，可以使用 [AdvantageScope](#) 可视化 swerve drive 的状态。下面的代码展示如何把一组 `SwerveModuleState` 对象发布到 `NetworkTables`。

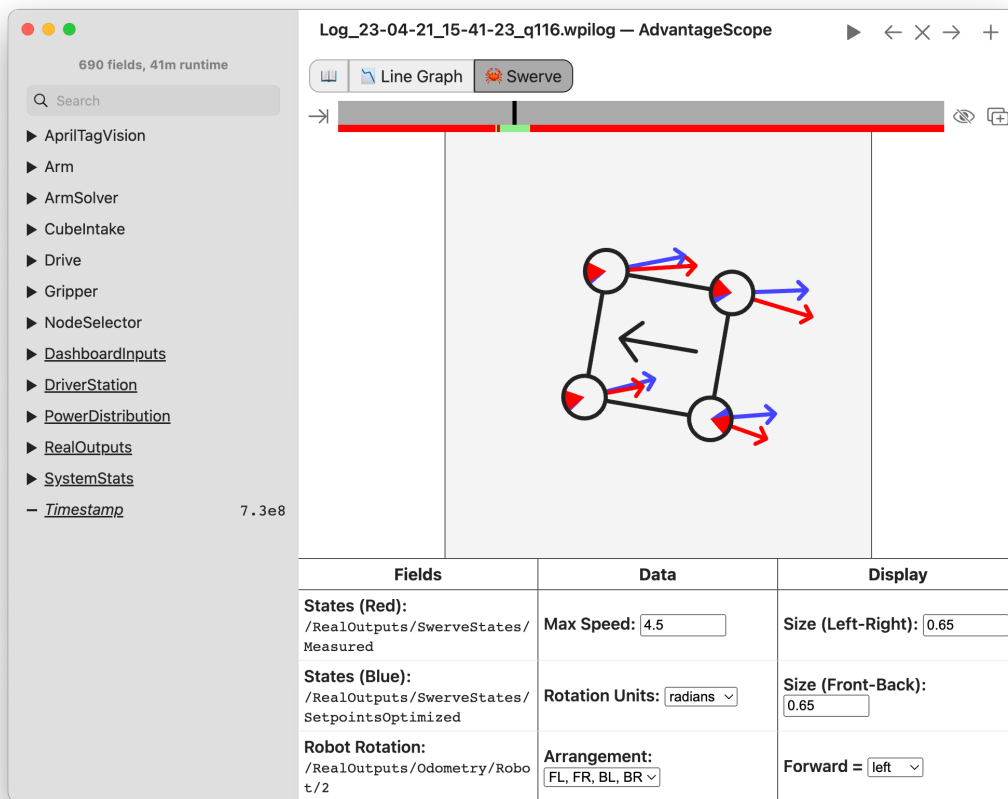
```

public class Example {
    private final StructArrayPublisher<SwerveModuleState> publisher;
    public Example() {
        // Start publishing an array of module states with the "/SwerveStates" key
        publisher = NetworkTableInstance.getDefault()
            .getStructArrayTopic("/SwerveStates", SwerveModuleState.struct).publish();
    }
    public void periodic() {
        // Periodically send a set of module states
        publisher.set(new SwerveModuleState[] {
            frontLeftState,
            frontRightState,
            backLeftState,
            backRightState
        });
    }
}

```

See the documentation for the [swerve](#) tab for more details on visualizing this data using AdvantageScope.

有关使用 AdvantageScope 可视化这些数据的更多信息，请参阅 [swerve](#) 标签页的文档。



Screenshot of an AdvantageScope window displaying a swerve visualization. / 显示 swerve 可视化视图的 AdvantageScope 窗口截图。

05 Swerve Drive Odometry / Swerve Drive 里程计

A user can use the swerve drive kinematics classes in order to perform [odometry](#). WPILib contains a `SwerveDriveOdometry` class that can be used to track the position of a swerve drive robot on the field.

用户可以使用 swerve drive 运动学类进行[里程计计算](#)。WPILib 提供 `SwerveDriveOdometry` 类，用于跟踪 swerve drive 机器人在场地上的位置。

NOTE: Because this method only uses encoders and a gyro, the estimate of the robot's position on the field will drift over time, especially as your robot comes into contact with other robots during gameplay. However, odometry is usually very accurate during the autonomous period.

说明： 由于这种方法只使用编码器和陀螺仪，对机器人场地位置的估计会随时间漂移；在比赛中机器人与其他机器人发生碰撞时尤为明显。不过，在自动阶段，里程计通常非常准确。

Creating the odometry object / 创建里程计对象

The `SwerveDriveOdometry` class constructor requires three mandatory arguments and one optional argument.

`SwerveDriveOdometry` 类的构造函数需要三个必选参数和一个可选参数。

The mandatory arguments are:

必选参数为：

- The kinematics object that represents your swerve drive (as a `SwerveDriveKinematics` instance)
- 表示 swerve drive 的运动学对象 (`SwerveDriveKinematics` 实例)。
- The angle reported by your gyroscope (as a `Rotation2d`)
- 陀螺仪报告的角度 (表示为 `Rotation2d`)。
- The initial positions of the swerve modules (as an array of `SwerveModulePosition`). In Java, this must be constructed with each wheel position in meters. It is important that the order in which you pass the `SwerveModulePosition` objects is the same as the order in which you created the kinematics object.
- swerve 模块的初始位置 (`SwerveModulePosition` 数组)。在 Java 中，构造每个车轮位置时必须使用米。传入 `SwerveModulePosition` 对象的顺序必须与创建运动学对象时的顺序一致。

The fourth optional argument is the starting pose of your robot on the field (as a `Pose2d`). By default, the robot will start at $x = 0$, $y = 0$, $\theta = 0$.

第四个可选参数是机器人在场地上的初始位姿 (pose, 以 `Pose2d` 表示)。默认情况下，机器人从 $x = 0$, $y = 0$, $\theta = 0$ 开始。

NOTE: 0 degrees / radians represents the robot angle when the robot is facing directly toward your opponent's alliance station. As your robot turns to the left, your gyroscope angle should increase. The `Gyro` interface supplies `getRotation2d` that you can use for this purpose. See [coordinate system](#) for more information about the coordinate system.

说明： 0 度 / 弧度表示机器人正对对方联盟站时的朝向。机器人向左转动时，陀螺仪角度应增大。`Gyro` 接口提供可用于此目的的 `getRotation2d`。有关坐标系的更多信息，请参阅坐标系章节。

```
// Locations for the swerve drive modules relative to the robot center.
Translation2d m_frontLeftLocation = new Translation2d(0.381, 0.381);
Translation2d m_frontRightLocation = new Translation2d(0.381, -0.381);
Translation2d m_backLeftLocation = new Translation2d(-0.381, 0.381);
Translation2d m_backRightLocation = new Translation2d(-0.381, -0.381);
// Creating my kinematics object using the module locations
SwerveDriveKinematics m_kinematics = new SwerveDriveKinematics(
    m_frontLeftLocation, m_frontRightLocation, m_backLeftLocation, m_backRightLocation
);
// Creating my odometry object from the kinematics object and the initial wheel positions.
// Here, our starting pose is 5 meters along the long end of the field and in the
// center of the field along the short end, facing the opposing alliance wall.
SwerveDriveOdometry m_odometry = new SwerveDriveOdometry(
    m_kinematics, m_gyro.getRotation2d(),
    new SwerveModulePosition[] {
        m_frontLeftModule.getPosition(),
        m_frontRightModule.getPosition(),
        m_backLeftModule.getPosition(),
        m_backRightModule.getPosition()
    }, new Pose2d(5.0, 13.5, new Rotation2d()));
```

Updating the robot pose / 更新机器人位姿

The `update` method of the odometry class updates the robot position on the field. The update method takes in the gyro angle of the robot, along with an array of `SwerveModulePosition` objects. It is important that the order in which you pass the `SwerveModulePosition` objects is the same as the order in which you created the kinematics object.

里程计类的 `update` 方法会更新机器人在场地上的位置。该方法接收机器人陀螺仪角度，以及 `SwerveModulePosition` 对象数组。传入这些对象的顺序必须与创建运动学对象时的顺序一致。

This `update` method must be called periodically, preferably in the `periodic()` method of a [Subsystem](#). The update method returns the new updated pose of the robot.

必须周期性调用这个 `update` 方法，最好在某个 [Subsystem](#) 的 `periodic()` 方法中调用。该方法返回更新后的机器人位姿。

```
@Override
public void periodic() {
    // Get the rotation of the robot from the gyro.
    var gyroAngle = m_gyro.getRotation2d();
    // Update the pose
    m_pose = m_odometry.update(gyroAngle,
        new SwerveModulePosition[] {
            m_frontLeftModule.getPosition(), m_frontRightModule.getPosition(),
            m_backLeftModule.getPosition(), m_backRightModule.getPosition()
        });
}
```

Resetting the Robot Pose / 重置机器人位姿

The robot pose can be reset via the `resetPosition` method. This method accepts three arguments: the current gyro angle, an array of the current module positions (as in the constructor and update method), and the new field-relative pose.

可以通过 `resetPosition` 方法重置机器人位姿。该方法接收三个参数：当前陀螺仪角度、当前模块位置数组（与构造函数和 `update` 方法中的格式相同），以及新的场地相对位姿。

IMPORTANT: If at any time, you decide to reset your gyroscope or wheel encoders, the `resetPosition` method MUST be called with the new gyro angle and wheel encoder positions.

重要： 如果你在什么时候重置陀螺仪或车轮编码器，必须使用新的陀螺仪角度和车轮编码器位置调用 `resetPosition` 方法。

NOTE: The implementation of `getPosition()` above is left to the user. The idea is to get the module position (distance and angle) from each module. For a full example, see here: [Java](#)

说明： 上面 `getPosition()` 的实现由用户自行完成，其作用是获取每个模块的位置（距离和角度）。完整示例见：[Java](#)。

In addition, the `getPoseMeters` method can be used to retrieve the current robot pose without an update.

此外，还可以使用 `getPoseMeters` 方法在不更新里程计的情况下获取机器人当前位姿。

06 Mecanum Drive Kinematics / 麦克纳姆驱动运动学

The `MecanumDriveKinematics` class is a useful tool that converts between a `ChassisSpeeds` object and a `MecanumDriveWheelSpeeds` object, which contains velocities for each of the four wheels on a mecanum drive.

`MecanumDriveKinematics` 类用于在 `ChassisSpeeds` 对象与 `MecanumDriveWheelSpeeds` 对象之间转换；后者包含麦克纳姆驱动四个车轮各自的速度。

NOTE: Mecanum kinematics uses a common coordinate system. You may wish to reference the [coordinate system](#) section for details.

说明： 麦克纳姆运动学使用共同的坐标系。有关详细信息，可参阅坐标系章节。

Constructing the Kinematics Object / 构造运动学对象

The `MecanumDriveKinematics` class accepts four constructor arguments, with each argument being the location of a wheel relative to the robot center (as a `Translation2d`). The order for the arguments is front left, front right, back left, and back right. The locations for the wheels must be relative to the center of the robot. Positive x values represent moving toward the front of the robot whereas positive y values represent moving toward the left of the robot.

`MecanumDriveKinematics` 类的构造函数接收四个参数，每个参数都是一个车轮相对于机器人中心的位置（以 `Translation2d` 表示）。参数顺序为左前、右前、左后、右后。车轮位置必须相对于机器人中心。x 为正表示朝机器人前方，y 为正表示朝机器人左方。

```
// Locations of the wheels relative to the robot center.
Translation2d m_frontLeftLocation = new Translation2d(0.381, 0.381);
Translation2d m_frontRightLocation = new Translation2d(0.381, -0.381);
Translation2d m_backLeftLocation = new Translation2d(-0.381, 0.381);
Translation2d m_backRightLocation = new Translation2d(-0.381, -0.381);
// Creating my kinematics object using the wheel locations.
MecanumDriveKinematics m_kinematics = new MecanumDriveKinematics(
    m_frontLeftLocation, m_frontRightLocation, m_backLeftLocation, m_backRightLocation
);
```

Converting Chassis Speeds to Wheel Speeds / 将底盘速度转换为车轮速度

The `toWheelSpeeds(ChassisSpeeds speeds)` method should be used to convert a `ChassisSpeeds` object to a `MecanumDriveWheelSpeeds` object. This is useful in situations where you have to convert a forward velocity, sideways velocity, and an angular velocity into individual wheel speeds.

应使用 `toWheelSpeeds(ChassisSpeeds speeds)` 方法将 `ChassisSpeeds` 对象转换为 `MecanumDriveWheelSpeeds` 对象。需要把前向速度、侧向速度和角速度转换为各车轮速度时，该方法很有用。

```
// Example chassis speeds: 1 meter per second forward, 3 meters
// per second to the left, and rotation at 1.5 radians per second
// counterclockwise.
ChassisSpeeds speeds = new ChassisSpeeds(1.0, 3.0, 1.5);
// Convert to wheel speeds
MecanumDriveWheelSpeeds wheelSpeeds = kinematics.toWheelSpeeds(speeds);
// Get the individual wheel speeds
double frontLeft = wheelSpeeds.frontLeftMetersPerSecond
double frontRight = wheelSpeeds.frontRightMetersPerSecond
double backLeft = wheelSpeeds.rearLeftMetersPerSecond
double backRight = wheelSpeeds.rearRightMetersPerSecond
```

译注：原文 Java 示例中四个 `double` 赋值语句缺少分号；此处按纯翻译要求保留原代码。

Field-oriented drive / 场地定向驾驶

[Recall](#) that a `ChassisSpeeds` object can be created from a set of desired field-oriented speeds. This feature can be used to get wheel speeds from a set of desired field-oriented speeds.

[回顾](#)：可以由一组期望的场地定向速度创建 `ChassisSpeeds` 对象。利用这一功能，可以根据一组期望的场地定向速度得到车轮速度。

```
// The desired field relative speed here is 2 meters per second
// toward the opponent's alliance station wall, and 2 meters per
// second toward the left field boundary. The desired rotation
// is a quarter of a rotation per second counterclockwise. The current
// robot angle is 45 degrees.
ChassisSpeeds speeds = ChassisSpeeds.fromFieldRelativeSpeeds(
    2.0, 2.0, Math.PI / 2.0, Rotation2d.fromDegrees(45.0));
// Now use this in our kinematics
MecanumDriveWheelSpeeds wheelSpeeds = kinematics.toWheelSpeeds(speeds);
```

Using custom centers of rotation / 使用自定义旋转中心

Sometimes, rotating around one specific corner might be desirable for certain evasive maneuvers. This type of behavior is also supported by the WPILib classes. The same `toWheelSpeeds()` method accepts a second parameter for the center of rotation (as a `Translation2d`). Just like the wheel locations, the `Translation2d` representing the center of rotation should be relative to the robot center.

在某些规避动作中，可能希望机器人绕某个特定角部旋转。WPILib 类也支持这种行为。同一个 `toWheelSpeeds()` 方法可接收第二个参数，表示旋转中心（类型为 `Translation2d`）。与车轮位置一样，表示旋转中心的 `Translation2d` 应相对于机器人中心。

NOTE: Because all robots are a rigid frame, the provided `vx` and `vy` velocities from the `ChassisSpeeds` object will still apply for the entirety of the robot. However, the `omega` from the `ChassisSpeeds` object will be measured from the center of rotation.

说明： 由于整个机器人是刚性框架，`ChassisSpeeds` 对象给出的 `vx` 和 `vy` 速度仍适用于整个机器人。不过，`ChassisSpeeds` 对象中的 `omega` 将以旋转中心为基准测量。

For example, one can set the center of rotation on a certain wheel and if the provided `ChassisSpeeds` object has a `vx` and `vy` of zero and a non-zero `omega`, the robot will appear to rotate around that particular wheel.

例如，可以将旋转中心设在某个车轮上；如果给定的 `ChassisSpeeds` 对象中 `vx` 和 `vy` 均为零，而 `omega` 非零，机器人看起来就会绕该车轮旋转。

Converting wheel speeds to chassis speeds / 将车轮速度转换为底盘速度

One can also use the kinematics object to convert a `MecanumDriveWheelSpeeds` object to a singular `ChassisSpeeds` object. The `toChassisSpeeds(MecanumDriveWheelSpeeds speeds)` method can be used to achieve this.

也可以使用运动学对象，将 `MecanumDriveWheelSpeeds` 对象转换为单个 `ChassisSpeeds` 对象。可使用 `toChassisSpeeds(MecanumDriveWheelSpeeds speeds)` 方法完成该转换。

```
// Example wheel speeds
var wheelSpeeds = new MecanumDriveWheelSpeeds(-17.67, 20.51, -13.44, 16.26);
// Convert to chassis speeds
ChassisSpeeds chassisSpeeds = kinematics.toChassisSpeeds(wheelSpeeds);
// Getting individual speeds
double forward = chassisSpeeds.vxMetersPerSecond;
double sideways = chassisSpeeds.vyMetersPerSecond;
double angular = chassisSpeeds.omegaRadiansPerSecond;
```

07 Mecanum Drive Odometry / 麦克纳姆驱动里程计

A user can use the mecanum drive kinematics classes in order to perform [odometry](#). WPILib contains a `MecanumDriveOdometry` class that can be used to track the position of a mecanum drive robot on the field.

用户可以使用麦克纳姆驱动运动学类进行[里程计计算](#)。WPILib 提供 `MecanumDriveOdometry` 类，用于跟踪麦克纳姆驱动机器人在场地上的位置。

NOTE: Because this method only uses encoders and a gyro, the estimate of the robot's position on the field will drift over time, especially as your robot comes into contact with other robots during gameplay. However, odometry is usually very accurate during the autonomous period.

说明： 由于这种方法只使用编码器和陀螺仪，对机器人场地位置的估计会随时间漂移；在比赛中机器人与其他机器人发生碰撞时尤为明显。不过，在自动阶段，里程计通常非常准确。

Creating the odometry object / 创建里程计对象

The `MecanumDriveOdometry` class constructor requires three mandatory arguments and one optional argument.

`MecanumDriveOdometry` 类的构造函数需要三个必选参数和一个可选参数。

The mandatory arguments are:

必选参数为：

- The kinematics object that represents your mecanum drive (as a `MecanumDriveKinematics` instance)
- 表示麦克纳姆驱动的运动学对象 (`MecanumDriveKinematics` 实例)。
- The angle reported by your gyroscope (as a `Rotation2d`)
- 陀螺仪报告的角度 (表示为 `Rotation2d`)。
- The initial positions of the wheels (as `MecanumDriveWheelPositions`). In Java, this must be constructed with each wheel position in meters.
- 车轮的初始位置 (以 `MecanumDriveWheelPositions` 表示)。在 Java 中，构造每个车轮位置时必须使用米。

The fourth optional argument is the starting pose of your robot on the field (as a `Pose2d`). By default, the robot will start at `x = 0`, `y = 0`, `theta = 0`.

第四个可选参数是机器人在场地上的初始位姿 (pose, 以 `Pose2d` 表示)。默认情况下, 机器人从 $x = 0$, $y = 0$, $\theta = 0$ 开始。

NOTE: 0 degrees / radians represents the robot angle when the robot is facing directly toward your opponent's alliance station. As your robot turns to the left, your gyroscope angle should increase. The `Gyro` interface supplies `getRotation2d` that you can use for this purpose. See [coordinate system](#) for more information about the coordinate system.

说明: 0 度 / 弧度表示机器人正对方联盟站时的朝向。机器人向左转动时, 陀螺仪角度应增大。`Gyro` 接口提供可用于此目的的 `getRotation2d`。有关坐标系的更多信息, 请参阅坐标系章节。

```
// Locations of the wheels relative to the robot center.
Translation2d m_frontLeftLocation = new Translation2d(0.381, 0.381);
Translation2d m_frontRightLocation = new Translation2d(0.381, -0.381);
Translation2d m_backLeftLocation = new Translation2d(-0.381, 0.381);
Translation2d m_backRightLocation = new Translation2d(-0.381, -0.381);
// Creating my kinematics object using the wheel locations.
MecanumDriveKinematics m_kinematics = new MecanumDriveKinematics(
    m_frontLeftLocation, m_frontRightLocation, m_backLeftLocation, m_backRightLocation
);
// Creating my odometry object from the kinematics object and the initial wheel positions.
// Here, our starting pose is 5 meters along the long end of the field and in the
// center of the field along the short end, facing the opposing alliance wall.
MecanumDriveOdometry m_odometry = new MecanumDriveOdometry(
    m_kinematics,
    m_gyro.getRotation2d(),
    new MecanumDriveWheelPositions(
        m_frontLeftEncoder.getDistance(), m_frontRightEncoder.getDistance(),
        m_backLeftEncoder.getDistance(), m_backRightEncoder.getDistance()
    ),
    new Pose2d(5.0, 13.5, new Rotation2d())
);
```

Updating the robot pose / 更新机器人位姿

The `update` method of the odometry class updates the robot position on the field. The update method takes in the gyro angle of the robot, along with a `MecanumDriveWheelPositions` object representing the position of each of the 4 wheels on the robot. This `update` method must be called periodically, preferably in the `periodic()` method of a [Subsystem](#). The `update` method returns the new updated pose of the robot.

里程计类的 `update` 方法会更新机器人在场地上的位置。该方法接收机器人陀螺仪角度, 以及一个表示机器人 4 个车轮位置的 `MecanumDriveWheelPositions` 对象。必须周期性调用该方法, 最好在某个 [Subsystem](#) 的 `periodic()` 方法中调用。该方法返回更新后的机器人位姿。

```
@Override
public void periodic() {
    // Get my wheel positions
    var wheelPositions = new MecanumDriveWheelPositions(
        m_frontLeftEncoder.getDistance(), m_frontRightEncoder.getDistance(),
        m_backLeftEncoder.getDistance(), m_backRightEncoder.getDistance());
    // Get the rotation of the robot from the gyro.
    var gyroAngle = m_gyro.getRotation2d();
    // Update the pose
    m_pose = m_odometry.update(gyroAngle, wheelPositions);
}
```

Resetting the Robot Pose / 重置机器人位姿

The robot pose can be reset via the `resetPosition` method. This method accepts three arguments: the current gyro angle, the current wheel positions, and the new field-relative pose.

可以通过 `resetPosition` 方法重置机器人位姿。该方法接收三个参数：当前陀螺仪角度、当前车轮位置，以及新的场地相对位姿。

IMPORTANT: If at any time, you decide to reset your gyroscope or encoders, the `resetPosition` method MUST be called with the new gyro angle and wheel positions.

重要： 如果你在任何时候重置陀螺仪或编码器，**必须**使用新的陀螺仪角度和车轮位置调用 `resetPosition` 方法。

NOTE: A full example of a mecanum drive robot with odometry is available here: [Java](#)

说明： 这里提供一个带里程计的麦克纳姆驱动机器人完整示例：[Java](#)。

In addition, the `getPoseMeters` method can be used to retrieve the current robot pose without an update.

此外，还可以使用 `getPoseMeters` 方法在不更新里程计的情况下获取机器人当前位姿。